

• OPENHARMONY EXPERIMENT

UART通信实验

基于开源鸿蒙的星闪开发板



九联星闪开发板



海思 WS63E 芯片



OpenHarmony OS



2026-05-08



深圳技术大学

实验内容

CONTENTS

01

UART通信原理

异步串行通信机制、数据帧结构、波特率同步、硬件连接要点

📶 核心原理讲解

02

实验环境准备

硬件连接配置、开发环境搭建、项目创建流程

🔧 环境搭建指南

03

printf打印实验

验证串口链路、调试输出、代码架构与编译烧录

➤ 基础实验验证

04

拓展实验

轮询收发与中断回调接收、DMA传输方式对比

📦 进阶实验探索

• CHAPTER 01

UART

通信原理

理解异步串行通信的核心机制
掌握数据帧结构与波特率同步

→ 包含动态原理演示

UART异步串行通信原理

Universal Asynchronous Receiver/Transmitter

• 动画演示



🕒 异步通信

不传时钟线，依靠预先约定的波特率在时间上对齐每一位

↔ 全双工

TX、RX各一根线可同时收发数据

🔗 共地要求

双方GND必须相连，否则参考电平不一致导致误判

UART数据帧结构

一帧数据的完整组成与传输时序

数据帧格式 (8N1)



常用参数配置

波特率 Baud Rate	115200 bps
数据位 Data Bits	8 bit
校验位 Parity	None (无)
停止位 Stop Bits	1 bit

参数写法: 8N1

关键特性

- ✓ 线路空闲状态
高电平 (逻辑1)
- ✓ 起始位同步
下降沿触发采样时刻
- ✓ 波特率一致性
双方必须一致, 否则乱码
- ✓ 行结束符
建议使用 \r\n 换行

实验环境 准备

硬件连接与开发环境配置

项目创建与源码管理



硬件连接



环境配置

实验硬件环境

九联星闪开发板硬件连接与调试配置

硬件连接示意图



九联星闪开发板
WS63E 芯片

UART_TX

MGPIO8 (S_MGPIO8)

UART_RX

MGPIO7 (S_MGPIO7)

GND

地线 (共地)

USB调试口

CH340 转串口

↓ TX → RX (交叉)

↑ RX → TX (交叉)



PC 端
串口助手

COM口

USB-SERIAL CH340

COM3

波特率

8N1 配置

115200

串口参数配置

波特率	115200
数据位	8 bit
校验位	None
停止位	1 bit
参数写法	8N1

硬件连接要点

- 交叉接线
TX接RX，RX接TX
- 共地连接
GND必须相连
- USB转串口
CH340驱动安装

项目创建流程

使用 make_app.py 创建与管理实验项目

推荐操作流程

Step 1



创建/启用工程

```
python3 applications/make_app.py -c uart_printf
```

- ✓ 在源码根目录执行
- ✓ 生成.c与BUILD.gn模板
- ✓ 自动修改三处配置



Step 2



替换示例源码

覆盖模板文件

- ✓ 用完整.c文件覆盖
- ✓ 用完整BUILD.gn覆盖
- ✓ 包含必要的include_dirs



重要： 只执行Step 1、不替换源码与BUILD.gn，一般无法通过编译或行为与文档不符

项目目录结构

applications/sample/wifi-iot/app/uart_printf/

uart_printf.c - 主程序源码

BUILD.gn - 构建配置

目标名：uart_printf_demo

> 常用命令速查

创建工程

```
make_app.py -c uart_printf
```

-c 参数

启用工程

```
make_app.py -e uart_printf
```

-e 参数

编译

```
make_app.py -b
```

-b 参数

全量编译

```
make_app.py -f
```

-f 参数

printf

打印实验

验证串口链路与调试输出

掌握OpenHarmony任务创建与延时



代码架构



编译运行

实验代码架构

uart_printf.c 程序结构与执行流程

程序执行流程

1 APP_FEATURE_INIT
特性初始化阶段注册入口

APP_FEATURE_INIT(UartPrintfEntry);



2 UartPrintfEntry
入口函数，创建任务

osThreadNew(UartPrintfTask, NULL, &attr)



3 UartPrintfTask
循环执行任务



printf
打印序号



喂狗
IoTWatchDogKick



延时
osDelay 1s

核心代码片段

```
static voidUartPrintfTask(void *arg)
{
    uint32_t seq = 1;
    while (1) {
        printf("Hello, Uart Printf! #u\r\n", seq);
        seq++;
        IoTWatchDogKick();
        osDelay(MS_TO_TICKS(1000));
    }
}
```

关键配置参数

任务栈大小	0x1000
任务优先级	25
系统Tick频率	100Hz
Tick周期	10ms
打印间隔	1秒

BUILD.gn 配置

```
static_library("uart_printf_demo") {
    sources = [ "uart_printf.c" ]
}
```

编译烧录与运行

从代码到运行的完整流程与预期结果

完整操作流程



编译

Compile

```
make_app.py -b
```

- ✓ 增量编译
- ✓ 或 `hb build -f`



固件

Firmware

```
out/.../ws63-liteos-app_all.fwpkg
```

- ✓ 生成fwpkg文件
- ✓ WSL可访问



烧录

Burn

```
BurnTool + WS63
```

- ✓ 选择COM口
- ✓ RST进入烧录



运行

Run

```
串口助手 115200
```

- ✓ 8N1配置
- ✓ 查看输出

> 预期运行结果

串口助手 - COM3

```
→ Hello, Uart Printf! #1
→ Hello, Uart Printf! #2
→ Hello, Uart Printf! #3
→ Hello, Uart Printf! #4
→ Hello, Uart Printf! #5
→ ...
```

**特征：**约每秒输出一行，序号递增

常见问题排查



串口无输出

- 确认COM口与波特率115200
- 确认应用已启用：`make_app.py -l`
- 确认烧录最新固件并复位



打印间隔不准

- 核对内核tick配置
- 调整TICK_PER_SEC参数



多应用冲突

- 用-x禁用无关应用
- 仅保留本实验目标

● EXPERIMENT COMPLETED

实验完成

通过本实验，您已掌握UART通信原理、OpenHarmony项目创建流程、
printf调试输出验证方法



基础实验

printf打印验证串口链路



拓展实验

轮询收发与中断回调



进阶探索

DMA传输与高级应用

 **参考文档：** 实验指导书第八、九节详细介绍轮询收发(uapi_uart_read/write)与中断回调接收(uapi_uart_register_rx_callback)的完整实现